



POD FINAL

Java Streams

Ejercicio 1

Dada la siguiente clase **Employee** (que no puede modificarse):

```
public class Employee {  
  
    private final String name;  
    private int age;  
  
    public Employee(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public int getAge() {  
        return age;  
    }  
}
```

Implementar el método **getRetiredEmployees** que, a partir de una colección de **Employee** y una edad límite, retorna una nueva lista con los nombres de los empleados que están en edad de jubilarse (esto es, tienen una edad mayor o igual a la edad límite recibida por parámetro).

```
private static List<String> getRetiredEmployeeList(List<Employee> employeeList, int age) {  
    ...  
}
```

Implementarlo utilizando únicamente **streams** y funcionalidades de **Java 1.8+** de forma que, con el siguiente programa de prueba:

```
public static void main(String[] args) {  
    List<Employee> employeesList = List.of(  
        new Employee("Foo", 50),  
        new Employee("Xyz", 60),  
        new Employee("Abc", 60),  
        new Employee("Bar", 75),  
        new Employee("Foo", 30)  
    );  
  
    // Ejemplo de invocación: se jubilan los que tengan 60 años o más  
    List<String> retiredEmployeesList =  
        getRetiredEmployees(employeesList, 60);  
    retiredEmployeesList.forEach(System.out::println);  
}
```

se obtenga la siguiente salida:

Bar
Abc
Xyz

Respuesta

```
private static List<String> getRetiredEmployeeList(List<Employee> employeeList, int age) {  
    return employeeList.stream().filter(employee → employee.getAge() >= age).map(Employee::getName).collect(Collectors.toList());  
}
```

Ejercicio 2 (Optional class)

A partir de clases citizen country y city. Implementar: `public static void printCountry(Citizen citizen) {}` utilizando `Optional` y getters.

```
class City {  
    private String cityName;  
  
    public City(String cityName) {  
        this.cityName = cityName;  
    }  
  
    public String getCityName() {  
        return cityName;  
    }  
}  
  
class Country {  
    private String countryName;  
    private City capitalCity;  
  
    public Country(String countryName, City capitalCity) {  
        this.countryName = countryName;  
        this.capitalCity = capitalCity;  
    }  
  
    public String getCountryName() {  
        return countryName;  
    }  
  
    public Optional<City> getCapitalCity() {  
        return Optional.ofNullable(capitalCity);  
    }  
}  
  
class Citizen {  
    private String name;  
    private Optional<Country> country;  
  
    public Citizen(String name, Country country) {  
        this.name = name;  
        this.country = Optional.ofNullable(country);  
    }  
  
    public String getName() {  
        return name;  
    }  
}
```

```

public Optional<Country> getCountry() {
    return country;
}
}

```

Implementar el método:

```

public static void printCountry(Citizen citizen) {
    String countryName = ...;
    System.out.println(countryName);
}

```

utilizando únicamente métodos de instancia de la clase Optional y getters de las clases involucradas de forma que, con el siguiente programa de prueba:

```

Citizen citizen = new Citizen("Juan Díaz", 30);
printCountry(citizen);
citizen.setCity(new City("BUE", new Country("ARG")));
printCountry(citizen);

```

se obtenga la siguiente salida:

Unavailable

ARG

Respuesta

```

private void printCountry(Citizen citizen) {
    System.out.println(citizen.getCountry().map(Country::getCountryName).orElse("Unavailable"));
}

```

Ejercicio 3

```

public class Usuario {
    private String id;
    private Vivienda vivienda;
    private String nombre;
    private int edad;
    // getters
}

class Vivienda {
    private Optional<String> barrio;
    private String ciudad;
    private String provincia;
    private String pais;
    // getters
}

```

Partiendo de una lista de Usuario llamada users y utilizando streams y funcionalidades de Java 8, indicar:

El número de usuarios por barrio para los usuarios que viven en la ciudad “Córdoba” del país “Argentina”. En caso de que el usuario no tenga un barrio de nido, indicar al barrio como "Desconocido".

Respuesta

```

users
    .stream()

```

```
.map(Usuario::getVivienda)
.filter(v → v.getPais().equals("Argentina") && v.getCiudad().equals("Córdoba"))
.collect(Collectors.groupingBy(v → v.getBarrio().orElse("Desconocido"), Collectors.counting()))
```

```
users
    .stream()
    .map(Usuario::getVivienda)
    .filter(v → v.getPais().equals("Argentina") && v.getCiudad().equals("Cordoba"))
    .collect(Collectors.toMap(v → v.getBarrio().orElse("Desconocido"), v → 1, Integer::sum));
```

La cantidad de usuarios menores de edad (18 años), agrupados por provincia, sabiendo que el nombre de las provincias de cada usuario puede tener cualquier combinación de mayúsculas y minúsculas.

```
users
    .stream()
    .filter(u → u.getEdad() < 18)
    .collect(Collectors.toMap(u → u.getVivienda().getProvincia().toLowerCase(), Collectors.counting()));
```

```
users
    .stream()
    .filter(u → u.getEdad() < 18)
    .collect(Collectors.toMap(u → u.getVivienda().getProvincia().toLowerCase(), v → 1, (a, b) → a + b));
```

Se quiere un único String con los nombres de usuarios en minúsculas, ordenados ascendentemente, sin repetidos y separados por comas.

```
users
    .stream()
    .map(u → u.getNombre())
    .sorted() // Por default ordena alfabéticamente (ascendentemente)
    .distinct()
    .collect(Collectors.joining(","));
```

Ejercicio 4

Analizar el siguiente código e indicar cual es el resultado del mismo:

```
final List <String> list = Arrays.asList("John", "Paul", "Ringo", "George");
int sum = 0;
list.forEach(e → {
    sum += e.length();
});
System.out.println(sum);
```

Respuesta

No compila, “variable used in lambda expression should be final or effectively final”

El integer te devuelve una nueva instancia todo el tiempo cuando la modificas, por eso falla.

```
// Solucion con streams
int total = list.stream().collect(Collectors.summingInt(String::length));
int total2 = list.stream().mapToInt(String::length).sum();

// Ejemplo con una clase random
final List <String> list = Arrays.asList("John", "Paul", "Ringo", "George");
Usuario us = new Usuario();
list.forEach(e → {
```

```

    us.setEdad(e.length());
});
System.out.println(us.getEdad());

// Ejemplo con un AtomicInteger
final List <String> list = Arrays.asList("John", "Paul", "Ringo", "George");
AtomicInteger sum = new AtomicInteger();
list.forEach(e → {
    sum.addAndGet(e.length());
});
System.out.println(sum.get());

```

Programación Concurrente - Threads, Thread Safety, Alternativas a la sincronización

Ejercicio 1

Indicar si es correcto que la variable counter tiene su lectura y escritura debidamente protegidas en el siguiente código (donde se ve toda la interacción de la clase con dicha variable).

```

public class Servlet {
    private Object lockCounterLock;
    private int counter;
    public void responseRequest(Request request) {
        ...
        synchronized (lockCounterLock) {
            ...
            counter ++;
        }
        ...
    }
    public synchronized int getCount(){
        return counter;
    }
}

```

Respuesta

No se sincronizan bajo la misma key. El metodo usa la key "this" y el responseRequest usa el lockCounterLock. Una posible alternativa es la siguiente:

```

public class Servlet {
    private Object lockCounterLock;
    private int counter;

    public void responseRequest(Request request) {
        ...
        synchronized (lockCounterLock) {
            ...
            counter ++;
        }
        ...
    }

    public int getCount(){
        synchronized (lockCounterLock) {
            return counter;
        }
    }
}

```

```
}  
}
```

Ejercicio 2

Clase sin setters, solo get de una variable final

Respuesta

Es una clase inmutable

Ejercicio 3

Suponiendo que una instancia del siguiente objeto Locker quiere usarse para realizar locks. La idea es que un usuario pide Lock mediante un texto y si no está pedido se devuelve true y si está se devuelve false. El objeto quiere utilizarse entre varios threads que ejecutan concurrentemente.

Indicar si el mismo tiene, o no, riesgos de utilizarse en dicha situación. Si el código puede fallar.

Indique como y cual puede ser una solución a dicho problema.

```
public class Locker {  
  
    private final Map<String, String> locks;  
  
    public Locker() {  
        locks = new ConcurrentHashMap<>();  
    }  
  
    public final boolean lock(final String path) {  
        if(!locks.containsKey(path)) {  
            locks.put(path, "locked");  
            return true;  
        }  
  
        return false;  
    }  
  
    public final void unlock(final String path) {  
        if(locks.containsKey(path)) {  
            locks.remove(path);  
        }  
    }  
  
}
```

Respuesta

Entra un thread, pregunta si contiene la key "hola". Como el mapa no la tiene, pasa a el put. Mientras hace el put (y no termina de ejecutarse), entra otro thread, pregunta si tiene la key "hola", como no la tiene pasa a el put.

Entonces ambos hacen put de la misma key y valor y ambos van a pensar que tienen el lock.

La solucion, es meterle un synchronized a nivel metodo directamente.

Ejercicio 4

Código donde String[] elements esta protegido en push y pop pero no en peek e isEmpty

Respuesta

No esta preparado para trabajar de manera concurrente. Si hacemos un peek o isEmpty y al mismo tiempo se ejecuta un push o un pop, podemos obtener inconsistencias.

Ejercicio 5

Para el siguiente código indique si el mismo es apto para utilizar en un ambiente concurrente. Justifique en ambos casos y en caso de no serlo proponga una solución.

```
public class StringBuffer {
    private String buffer;
    private int length;

    public StringBuffer() {
        this("");
    }

    public StringBuffer(final String buffer) {
        this.buffer = buffer;
        length = buffer.length();
    }

    public StringBuffer append(final String suffix) {
        return new StringBuffer(buffer + suffix);
    }

    public int getLength() {
        return length;
    }

    @Override
    public String toString() {
        return buffer;
    }
}
```

Respuesta

Se puede usar en un ambiente concurrente. El objeto es inmutable: las variables son privadas, no se modifican y no hay setters disponibles.

Como el objeto es inmutable, hacer un append retorna una nueva instancia del objeto (que es un objeto inmutable).

Ejercicio 6

Porque usar **synchronized** en definición de metodo no es muy recomendable

Respuesta

Esto conduce a la sincronización de todo el método, lo que significa que ningún otro hilo podrá acceder a un bloque sincronizado con la misma instancia (se bloquea). Al utilizar **synchronized** de esta manera, especialmente si los métodos son extensos, estoy limitando la eficiencia del resto del sistema en términos de concurrencia, ya que estaré bloqueando a todos los hilos que deseen trabajar con esta instancia. Lo más recomendable sería aplicar la sincronización solo a las variables y partes del código que lo requieran, a menos que se presente un caso excepcional donde el método completo lo necesite o sea breve y sencillo. Es importante tener en cuenta que al sincronizar un método completo se diluye en cierta medida el propósito de la programación concurrente, ya que cada cliente debe esperar a que todos los que llegaron antes terminen antes de poder ejecutar su tarea, convirtiendo el servicio en serial en lugar de concurrente.

gRPC - Introducción a sistemas distribuidos, Communication Patterns, Streaming

Como funciona gRPC

RPC: protocolo que especifica el llamado de ejecución de funciones remotas sin tener que preocuparse por la comunicación entre el cliente y el servidor.

gRPC es un sistema de comunicación vía RPC open source desarrollado inicialmente por Google que apunta a ser:

- Multiplataforma
- Multilenguaje
- De alto rendimiento
- De uso general
- Open Source

Componentes (skeleton, stubs, etc)

- Stub: funciona de proxy a un objeto remoto en el cliente. Implementa los mismos métodos que la interfaz remota. Su objetivo es recibir las invocaciones del cliente y pasarlas por la red (el método invocado y los parámetros) y esperar los resultados para leerlos y transmitirlos al cliente.
- Skeleton: se crea en el servidor, se encarga de recibir el pedido del cliente, interpretarlo, obtener los parámetros y llamar al objeto remoto con los mismos. Una vez que recibe la respuesta la transmite al cliente. Aislado de esta manera al objeto remoto de la forma de comunicación.

Diferencias entre los tipos de stub

Para uso en el cliente el Middleware provee 3 versiones de stub para realizar llamados a los servicios del server.

- BlockingStub: Stub sincrónico
- Stub: Stub asincrónico con observers para procesar
- FutureStub: Stub asincrónico que retorna Future

¿Cual es la diferencia entre repeated y stream?

Streaming y repeated son dos maneras de enviar más de un elemento en la comunicación.

- **Repeated**: es un modificador para un campo dentro de un mensaje.
- **Stream**: es un modificador para un mensaje (de entrada y/o salida).

Se pueden intercambiar, pero no son lo mismo. Técnicamente, **repeated** se usa para listas que se obtienen inmediatamente y **stream** se usa más cuando se tarda en recuperar los elementos (te las voy a mandar en el tiempo).

Indicar cual es la función del Servant

El servant es el objeto encargado de realizar "realmente" las operaciones del servicio. Para ello reside en el servidor, implementa la interfaz remota (o interfaces) y ejecuta las operaciones del servicio. Más allá de que implementa una interfaz remota, su ejecución es local y no cambia si se lo llamó remotamente o no.

APIs

Qué significa cuando se indica que una API es basada en eventos e indicar algún ejemplo de este tipo de API

Una API basada en eventos permite emitir y consumir eventos. Para eso, tiene un mecanismo que permite a un consumidor suscribirse y los medios para entregar eventos a los consumidores que están suscritos. Las APIs y/o servicios habilitados para emitir eventos (productores) se conectan a un intermediario (Kafka o RabbitMQ por ejemplo) permitiéndole a los clientes suscribirse a un canal de interés. Eventualmente, cuando se produce un evento de interés, se desencadena un flujo de datos hacia un cliente que está esperando dichos datos para poder procesarlos. Si la comunicación fuera bidireccional (web socket), el cliente podría publicar eventos en el backend a través de la API basada en eventos.

Un ejemplo de este tipo de API puede ser un web hook API, conocido también como "reverse API".

Indicar qué significa que un cliente utilice la estrategia de server side pushing para obtener la respuesta de un servicio remoto de manera no bloqueante.

Server side pushing (SSE) facilita la comunicación asincrónica entre servidores y clientes para aplicaciones web haciendo uso del protocolo HTTP. La idea es que los servidores envíen mensajes o eventos de manera unidireccional a los clientes, actualizándolos de forma asíncrona.

SSE elimina la necesidad de realizar polling o long-polling (requests periódicos), simplificando el proceso de comunicación y permitiendo que el cliente se actualice constantemente al haber nuevos datos disponibles.

Sin embargo, SSE trae consigo dos desventajas principales:

- Limitaciones en el formato de datos: Dado que SSE está restringido a transportar mensajes en formato UTF-8, no se pueden enviar datos binarios.
- Cuando no se utiliza sobre HTTP/2, otra limitación es el número restringido de conexiones concurrentes por navegador. Solo se permiten seis conexiones SSE concurrentes abiertas en todo momento.

Client-side polling

Client Polling es una técnica implementada en el cliente. La idea es enviar periódicamente solicitudes al servidor en busca de actualizaciones, obteniendo respuestas de manera no bloqueante.

Los datos que se obtienen no son en tiempo real; precisamente, se solicitan en intervalos fijos de tiempo. Es decir, los clientes envían solicitudes incluso cuando no hay actualizaciones en el servidor.

Hoy esta técnica ya no se usa. Para mas info ver long polling o web sockets.

→ Adicionalmente estudiaría la parte de REST y de GraphQL

Sistemas distribuidos

Ejercicio 1

De qué manera se puede implementar tolerancia a fallos en cuanto a la pérdida de datos en un sistema de store distribuidos

Respuesta

Se puede implementar replicando la data. De esta manera, se mantienen múltiples copias de los mismos en los nodos. Si algún nodo llegara a caerse o por algún motivo no puede accederse a los datos que contiene, sabemos que estarán disponibles y replicados en otros nodos.

Ejercicio 2

Indicar qué significa que un sistema de store cumpla con el modelo de consistencia eventual

Respuesta

Consistencia eventual significa que existe la posibilidad de leer un dato de un nodo que no esté actualizado (stale). Sin embargo, pasado cierto tiempo sin una escritura, todas las lecturas de un mismo dato para distintos nodos deberían devolver el mismo valor.

En otras palabras, consistencia eventual indica que un sistema puede no ser consistente en todo momento, pero eventualmente llegará a un estado consistente si no se realizan más actualizaciones.

Ejercicio 3

Explicar qué implica la capacidad de elasticidad que puede tener un clúster de trabajo o store distribuido

Respuesta

Es la capacidad de agregar o quitar nodos en un cluster sin tener que detener el mismo (on demand).

En principio, si se agrega un nodo al cluster, el cluster debe detectar el nuevo nodo, asignarle las particiones primarias y de backup, coordinar la transferencia de esas particiones y marcarlo como disponible para ser utilizado en las próximas operaciones.

Eliminar un nodo del clúster puede ser una tarea más compleja, ya que, en principio, debemos asegurarnos de que el nodo no contenga datos actualizados que se perderían si se elimina sin haberse actualizado en los otros nodos.

Ejercicio 4

Explicar particionado y replicación

Respuesta

Particionado: es el proceso de dividir los datos de un sistema en unidades más pequeñas e independientes. Cada una de esas unidades se suele llamar partición. Luego podemos asignar una o más particiones (distintas) a un nodo, donde el mismo será responsable de atender las solicitudes (lectura o escritura) de los datos asociados. El particionado de los datos trae 3 beneficios si se hace correctamente: fault tolerance, scalability y performance.

Replicación: La replicación es un proceso en el que se crean múltiples copias de los datos existentes para almacenarlos en los nodos del sistema distribuido. Cada una de estas copias se llama réplica. El principal beneficio de la replicación es el aumento de la tolerancia a fallos. Como los mismos datos se almacenan de forma redundante en varias máquinas, todo el sistema puede tolerar la falla de una o más de ellas, ya que los usuarios pueden acceder a los datos desde cualquier nodo que funcione.

→ Adicionalmente, estudiaría la parte de Sistemas Distribuidos y nada mas

Sistemas distribuidos (Hadoop y Hazelcast)

Ejercicio 1

En Hazelcast por defecto ¿Cómo se selecciona al nodo dueño de una partición?

Respuesta

Para la coordinación, asignación de trabajo y particiones se selecciona al nodo de mayor antigüedad del cluster como coordinador. El coordinador posee una tabla de particiones donde dice qué nodo es owner/backup de qué particiones, pero la distribuye entre todos. O sea, al llegar/desaparecer un nodo la actualiza y la vuelve a distribuir. En caso de que el coordinador se caiga se selecciona al siguiente nodo más viejo como nuevo coordinador y se recalcula la tabla de particiones.

Ejercicio 2

Explique el concepto de Split Brain. ¿Puede ocurrir utilizando el framework Hazelcast? Si es así, ¿cómo?

Respuesta

Dado un cluster se da un split brain cuando por problemas de comunicación el mismo se subdivide en partes autónomas. Esto puede causar graves fallas, porque los clusters siguen funcionando pudiendo recibir actualizaciones que conflictúan (por ejemplo reciben 2 valores distintos para la misma key) el posterior mergeo entre ambos. Hazelcast provee algunos mecanismos para solucionar esto: se elige entre los líderes de ambos clusters el mas viejo como nuevo líder, y por default se mergea el cluster mas chico al mas grande, aunque esto se puede definir. En el mientras tanto, se le puede indicar a Hazelcast que si el cluster tiene menos de una cierta cantidad de nodos, no se puede operar con el mismo y lanza una SplitBrainException.

Hazelcast - Map Reduce

Explicar cuál es la función del Combiner en el paradigma de programación distribuido MapReduce.

El Combiner siempre opera entre el Mapper y el Reducer, siendo su uso opcional. Tiene sentido emplearlo cuando la salida del Mapper es considerablemente extensa. Si no lo hiciéramos y esta gran cantidad de datos llegara directamente al Reducer, se produciría una congestión en la red. El Combiner, suele ser similar al Reducer, es reutilizable y trabaja de a chunks emitiendo resultados parciales.

Para que sirve reset() en el combiner

Se instancia uno por cada clave. El combiner al trabajar de a chunks, cuando devuelve su resultado parcial (finalizeChunk), hace uso de reset para volver a comenzar si es que aparece otro.

Ejercicio MapReduce 1

Supongamos un dataset de árboles de una ciudad que modela sus datos de la siguiente forma:

- id: identificador autogenerado
- barrio: Nombre del barrio donde se encuentra el árbol.
- calle: Nombre de la calle donde se encuentra el árbol
- especie: Especie del árbol

Suponiendo que se quiere utilizar un proceso mapreduce (que puede ser una cadena de procesos). En este proceso se recibe en el mapper inicial el id del árbol como clave y el resto de los datos del registro como valor. Se pide sin codificar, indique en palabras, para cada job en la cadena:

1. ¿Cuáles serían el comportamiento del Mapper?
2. ¿Cuál es la clave y el valor que emite mapper?
3. ¿Cuál sería el comportamiento del Reducer?
4. ¿Cuál es la clave y el valor que emite Reducer?

Indicar si hay que hacer un procesamiento posterior de los resultados. Para el caso en que se quiera resolver la siguiente consulta: *"El top ten de barrios con mayor cantidad de especies distintas ordenado descendente por cantidad."*

Ejemplo de resolución

```
function map(id, arbol) {  
  emit(arbol.barrio, arbol.especie);  
}
```

```
function reduce(barrio, values) {  
  let result = new Set<String>();  
  for especie in values {  
    if !result.contains(especie) {  
      result.add(especie)  
    }  
  }  
}
```

```
return (barrio, result.length());  
}
```

La idea es que por cada árbol, se emite un par clave-valor donde la clave es el barrio del árbol, y el valor es un arreglo con su un solo elemento; su especie.

Por ejemplo:

- 1 ⇒ ("nuñez", "oak")
- 2 ⇒ ("palermo", "oak")
- 3 ⇒ ("nuñez", "gomero")
- 4 ⇒ ("palermo", "oak")

Luego la operación de reduce tomará para cada clave (barrio), todas las especies emitidas, y contará cuantas especies distintas hay, para devolver un clave valor (barrio, cantidad_de_especies_distintas). Por ejemplo:

```
reduce("nuñez", ["oak", "gomero"]) → ("nuñez", 2)  
reduce("palermo", ["oak", "oak"]) → ("palermo", 1)
```

Por último, se requiere un post-procesamiento de los resultados, ya que estas tuplas deben ser ordenadas por la cantidad y luego se debe tomar las top 10.

Ejercicio MapReduce 2

Mapper recibe: [id, actividad departamento provincia], resolver consulta:

Para cada job en la cadena

- ¿Cuáles serían el comportamiento del Mapper?
- ¿Cuál es la clave y el valor que emite Mapper?
- ¿Cuál sería el comportamiento del Reducer?
- ¿Cuál es la clave y el valor que emite Reducer?
- Indicar si hay que hacer un procesamiento posterior de los resultados

Para el caso en que se quiera resolver la siguiente consulta:

Nombres de departamentos que aparecen en al menos "n" provincias, ordenado descendientemente por el numero de apariciones

[1, 'caca', 14, 'Formosa'] → [14, 'Formosa']

[2, 'pepe', 14, 'Chaco'] → [14, 'Chaco']

[3, 'pepe', 13, 'Chaco'] → [13, 'Chaco']

Mapper:

- Por cada registro, pone como clave el departamento y como valor la provincia
 - [id, actividad departamento provincia] → [departamento, provincia]
- El reducer
 - Si asumimos que un departamento puede estar una unica vez en una provincia: sumamos 1 por cada valor
 - Por cada provincia, sumo uno
 - Si asumimos que un departamento puede estar varias veces en una provincia: agrupa en un set las distintas provincias para evitar repetidos
 - Por cada provincia, la meto en el set
- Collator: filtrar los que no cumplen con n provincias y ordenar por numero de apariciones

Ejercicio MapReduce 3

Supongamos un dataset de movimientos de un aeropuerto (despegues y aterrizajes) que modela sus datos de la siguiente forma:

- **id**: identificador autogenerado
- **tipo**: Despegue o Aterrizaje.
- **origen**: Código del aeropuerto del cual despegó el vuelo.
- **destino**: Código del aeropuerto en el cual aterrizó el vuelo.

Suponiendo que se quiere utilizar un proceso map reduce (que puede ser una cadena de procesos).

En este proceso se recibe en el mapper inicial el id del movimiento como clave y el resto de los datos del registro como valor.

Se pide sin codificar, indique en palabras

Para cada job en la cadena

- ¿Cuáles serían el comportamiento del Mapper?
- ¿Cuál es la clave y el valor que emite Mapper?
- ¿Cuál sería el comportamiento del Reducer?
- ¿Cuál es la clave y el valor que emite Reducer?
- Indicar si hay que hacer un procesamiento posterior de los resultados

Para el caso en que se quiera resolver la siguiente consulta:

Top 10 aeropuertos destino con mayor cantidad de despegues que tienen como origen al aeropuerto EZE, ordenado descendente por cantidad de despegues

Respuesta

Escribo un pseudocódigo para ayudar a la explicación:

```
function map(id, tipo, origen, destino) {  
  if (origen == "EZE" && tipo == "Despegue") {  
    emit(destino, 1);  
  }  
}  
  
function reduce(destino, values) {  
  let total = 0;  
  for elem in values {  
    total += elem;  
  }  
  
  return (destino, total);  
}
```

El mapper se fija, por cada movimiento, si el movimiento tiene origen en Ezeiza y es un despegue. Si lo es, entonces emite un par clave-valor donde la clave es el destino del movimiento, y el valor es 1. Por ejemplo:

```
(id, tipo, origen, destino) ⇒ (lo que emite el mapper)  
map(0, "Despegue", "EZE", "ABC") ⇒ [("ABC", 1)]  
map(1, "Aterrizaje", "EZE", "ABC") ⇒ [] (nada)  
map(2, "Despegue", "MIA", "EZE") ⇒ []  
map(3, "Aterrizaje", "MIA", "EZE") ⇒ []  
map(4, "Despegue", "EZE", "DEF") ⇒ [("DEF", 1)]  
map(5, "Aterrizaje", "EZE", "DEF") ⇒ []  
map(6, "Despegue", "EZE", "ABC") ⇒ [("ABC", 1)]
```

El reducer luego toma, por cada clave distinta emitida por los mappers, todos sus valores, y simplemente los suma, emitiendo un nuevo par clave-valor que consiste del destino como clave, y la suma como valor. Siguiendo el ejemplo:

```
reduce("ABC", [1, 1]) ⇒ ("ABC", 2)  
reduce("DEF", [1]) ⇒ ("DEF", 1)
```

Por último, se requiere post-procesamiento. Estas tuplas contienen, para cada aeropuerto, la cantidad de despegues provenientes de Ezeiza hacia dicho aeropuerto. Debemos entonces ordenarlas descendente por cantidad de despegues, y quedarnos con el aeropuerto de las top 10 tuplas.

Ejercicio MapReduce 4

Mapper recibe: [id, estación línea fechaHora pasajeros], resolver consulta: estacion con mas pasajeros de cada linea para 2021, indicando estacion, linea y pasajeros. Orden alfabetico por linea.

Respuesta

mapper emite [linea, [estacion, pasajeros]], solo si la fecha es el 2021. sino no emite nada

reducer suma los pasajeros de cada estacion

reducer emite [linea, [estacion, totalPasajeros]]

post procesamiento: para cada linea, elegir la estacion donde el totalPasajeros sea el mayor valor y ordenar

Ejercicio MapReduce 5

Dentro de un sistema se reciben las notificaciones de los retweets realizados para una campaña a partir de un criterio de búsqueda (cierto hashtag, por ejemplo). Al terminar la misma, las notificaciones obtenidas son enviadas a un sistema map-

reduce para su procesamiento y obtener estadísticas. El proceso entonces recibe en el mapper:

- Como clave el id del retweet.
- Como valor los siguientes datos:
 - Screen name del usuario que realizó el retweet.
 - Id del tweet original retweeteado.
 - Screen name del autor del tweet.
 - Cantidad de seguidores del autor del tweet.
 - El país donde se realizó el tweet.
 - Cantidad de seguidores del que realizó el retweet.
 - Likes del tweet al momento de realizarse el retweet.
 - Fecha del retweet.
 - Un set con los hashtags del tweet.

Sin codificar, indique:

- Cuál es el procesamiento en el mapper y qué clave y valor emite.
- Cuál es el procesamiento y el valor de respuesta del reducer.
- Si se requiere algún procesamiento posterior al reducer para obtener la respuesta final.

Para la siguiente query:

El ranking de tweets por su cantidad de likes (Aclaración: un retweet no tiene likes, solo el tweet original y supongamos que no se puede "unlikear").

Respuesta

1. El mapper emite un par clave valor cuya clave es el id del tweet original y la cantidad de likes.
2. El reducer se encarga de guardar en una variable la mayor cantidad de likes de un tweet. Finalmente, emite el id del tweet original junto con un valor que representa el total de likes que recibió.
3. El post procesamiento es realizado por un Collator que se encarga de ordenar de mayor a menor la cantidad de likes.

Mapper recibe: [id retweet, user_name_rt id_tweet_og user_name_og followers_og pais followers_rt likes fecha hashtags], **resolver consulta:** top 10 usuarios que llegaron a mas gente con sus tweets, a partir de los retweets del mismo. **Comportamiento del mapper por cada mensaje recibido emite.**

Aclaraciones: no importa repetidos, no tomar en cuenta seguidosres de auto original.

Ejercicio MapReduce 6

Mapper recibe: [dd/mm/yyyy-ciudad valorMontoVenta], **resolver consulta:** monto diario de ventas promedio para cada año por ciudad (365 días por año). **Comportamiento del mapper.** Por cada dato, parsea el año y la ciudad en la clave.

⇒ Si se necesita orden aplica el post procesamiento

Ejercicio MapReduce 7

Una empresa que distribuye sus productos en distintas ciudades del mundo cuenta con datos correspondientes a las ventas que se registraron durante cierto período en las mismas. Cada valor del conjunto tiene la siguiente estructura: "dd/mm/yyyy-ciudad" valorMontoVenta.

Ejemplo: '10/10/2015Granada' 4000.30 significa que el 10 de Octubre del 2015 en la ciudad de Granada las ventas fueron de 4000.30 pesos.

Y hay un "registro" único para cada par día y ciudad.

Sin codificar, indique en palabras:

- ¿Cuáles serían el comportamiento del Mapper y la clave y el valor que emite?
- ¿Cuál sería el comportamiento del Reducer para obtener el valor final?

Para la siguiente query implementada en utilizando Map Reduce: calcule el monto promedio de ventas diario para cada año y ciudad.

1. El mapper se encarga de emitir un par clave valor cuya clave es un par con el nombre de la ciudad y el año y cuyo valor es el monto de ventas.
2. El reducer hace la suma de los montos de ventas de cada ciudad en un año. Luego, el reducer emite (ciudad-año, suma/365).

Para la siguiente query implementada en utilizando Map Reduce: calcule para cada ciudad cual fue el año con mayor monto de ventas.

1. El mapper se encarga de emitir un par clave valor cuya clave es el nombre de la ciudad y cuyo valor es un par con el año y el monto de ventas.
2. El reducer guarda un mapa cuya clave es el año y cuyo valor es la suma de los montos de ventas de dicho año. Luego, el reducer emite el año con el mayor monto de ventas..

NoSQL

¿Cuál es la restricción que presenta Cassandra a la hora de realizar filtros en sus consultas, por que existe esa restricción y qué desafío se presenta a partir de esta restricción?

A la hora de realizar consultas, un filtro se puede aplicar a la pk. Esta restricción existe para poder leer rápidamente ya que ese nodo sabe donde encontrar ese registro y en que nodo. Entonces, el desafío que se presenta, es que si yo necesito querear por diferentes campos, tengo que armar una tabla por query con dichos campos

¿Por que HBASE tiene consistencia de escritura a pesar de ser distribuido?

Esto se debe a la replicación master/slave que maneja HBASE, donde un cliente solo puede leer/escribir desde el master, lo que garantiza tener la última escritura.

La lectura no necesariamente pasa por el master, puede pasar por un slave.

¿Por que el tratamiento del valor en REDIS es más avanzado que en otras key-value?

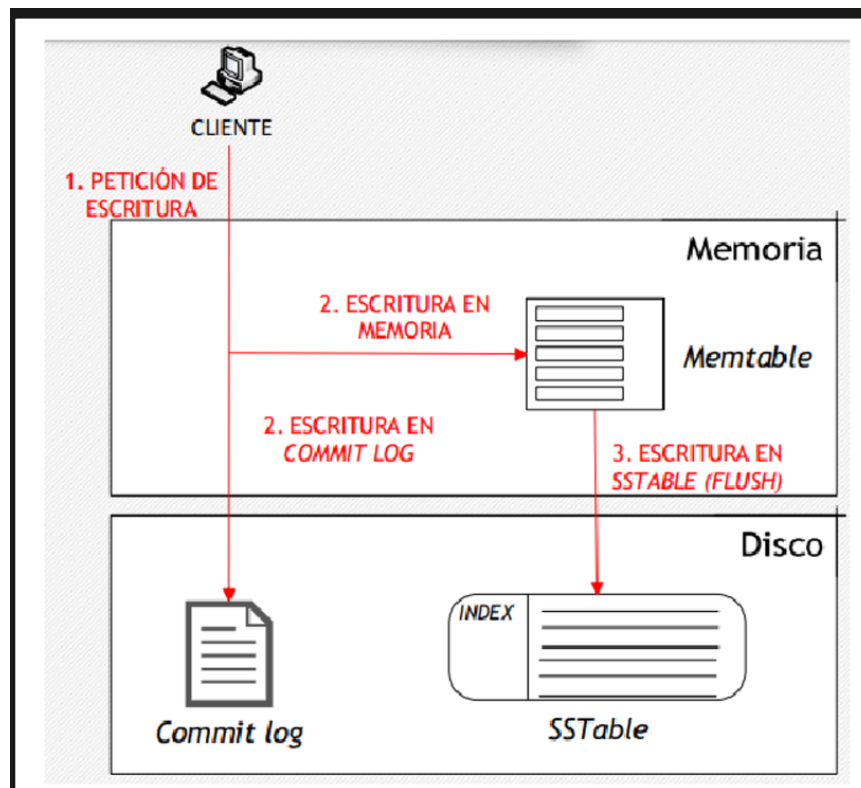
REDIS le agrega semántica a sus valores a diferencia de las key-value tradicionales. Sin embargo, conserva ciertas características de estas ultimas: sus valores son opacos, lo que imposibilita operar sobre los mismos.

¿Cuáles son los 3 componentes del teorema CAP y que indica el teorema de la relación entre ellos para sistemas distribuidos?

- **Consistency:** Indica que todos los nodos devuelvan el último dato escrito
- **Availability:** Garantiza que todo request será inmediatamente respondido (puede ser con error)
- **Partition tolerance:** El sistema sigue funcionando a pesar de que está particionado y alguno de los nodos falla o pierde información.

El teorema indica que no se pueden tener las 3, se debe elegir 2. Como estamos hablando de herramientas que escalan horizontalmente necesitamos partición. Por lo que hay que elegir entre Consistencia o Disponibilidad. En general (no siempre) las NoSQL eligen disponibilidad, por lo que eligen aplicar consistencia eventual.

Explicar la estrategia de Cassandra realiza la escritura y replicación de un dato



Read → se mira el consistency level

Write → se mira el replication factor

2023 2C 1

Pregunta 1:

¿Qué es lo que se indica al decir que una base de datos provee consistencia eventual?

Respuesta

Promete que dado una escritura, si no hay una nueva actualización, todas las réplicas coincidirá eventualmente en el último valor escrito.

La cantidad de réplicas afecta en la velocidad. Menos réplicas, la distribución es más rápida, pero el riesgo de pérdida de datos es mayor

Pregunta 2:

¿Las bases de datos Columnares por su manera de guardar son muy eficientes para qué tipo de consultas?

Respuesta

Una base de datos columnar está optimizada para leer columnas de la información (en contrapartida de leer filas).

- El principal uso es para hacer queries analíticas, que se ven beneficiadas por este tipo de guardado.
- Reducen mucho los requerimientos de accesos de I/O y la cantidad de data a levantar.
- Permiten una mejor compresión, ya que la data por archivos es más homogénea.
- I/O se reduce ya que solo se leen las columnas requeridas para resolver la respuesta (en vez de realizar un "full scan").
- Se diseñan fácil para escalar horizontalmente en hardware barato.

Pregunta 3:

Para el siguiente código indique si el mismo es apto para utilizar en un ambiente concurrente. Justifique en ambos casos y en caso de no serlo proponga una solución.

```

public class Friend {

    private final String name;

    public Friend(String name) {
        this.name = name;
    }
  
```



```

    public String getName() {
        return this.name;
    }

    public synchronized void bow(Friend bower) {
        System.out.format("%s: %s has bowed to me!%n", this.name, bower.getName());
        // ----- LOS DOS THREADS QUEDAN ACA
        bower.bowBack(this);
    }

    public synchronized void bowBack(Friend bower) {
        System.out.format("%s: %s has bowed back to me!%n", this.name, bower.getName());
    }
}

```

```

Friend f1 = new Friend();
Friend f2 = new Friend();

```

Thread 1:

f1.bow(f2)

thread 1 toma el lock de f1

"F1: F2 has bowed to me"

Thread 2:

f2.bow(f1)

thread 2 toma el lock de f2

"F2: F1 has bowed to me"

Thread 1

Quiere llamar f2.bowBack(f1)

El lock de f2 lo tiene el thread 2 (en bow), entonces al llamar a f2.bowBack(f1) se bloquea

Thread 2

Quiere llamar f1.bowBack(f2)

El lock de f1 lo tiene el thread 1 (en bow), entonces al llamar a f1.bowBack(f2) se bloquea

Deadlock

Solucion

```

public class Friend {

    private static final Object lock = "lock";
    private final String name;

    public Friend(String name) {
        this.name = name;
    }
}

```

```

public String getName() {
    return this.name;
}

public void bow(Friend bower) {
    synchronized (lock) {
        System.out.format("%s: %s has bowed to me!\n", this.name, bower.getName());
        bower.bowBack(this);
    }
}

public synchronized void bowBack(Friend bower) {
    synchronized (lock) {
        System.out.format("%s: %s has bowed back to me!\n", this.name, bower.getName());
    }
}
}

```

Pregunta 7:

Supongamos un sitio de reviews de películas que modela sus datos:

- id: identificador autogenerado
- título: nombre de las películas.
- director: Lista de los directores (cada uno un string de nombres y apellidos)
- Protagonistas: Lista de protagonistas (cada uno un string de nombres y apellidos).
- Género: Texto que indica el género principal de la película.
- Clasificación: valor que indica si la película es apta para todo público o tiene alguna restricción.
- Fecha de estreno.
- Taquilla: Recaudación total de la película.
- Reviews: Lista de reviews donde cada review tiene:
 - username: identificador del usuario.
 - rating: calificación numérica 1 a 10.
 - comentario: texto libre para indicar razones de la calificación.
 - likes: cantidad de likes dados por otros usuarios al review.

Tener en cuenta que el nombre de una película puede aparecer muchas veces ya sea por remakes o por películas que simplemente se llaman igual.

Suponiendo que se quiere utilizar un proceso map reduce (que puede ser una cadena de procesos). En este proceso se recibe en el mapper inicial el id de la película como clave y el resto de los datos del registro como valor.

Se pide sin codificar, indique en palabras

Para cada job en la cadena:

1. ¿Cuáles serían el comportamiento del *Mapper*?
2. ¿Cuáles es la clave y el valor que emite mapper?
3. ¿Cuál sería el comportamiento del *Reducer*?
4. ¿Cuáles es la clave y el valor que emite reducer?
5. Indicar si hay que hacer un procesamiento posterior de los resultados.

Para:

Indicar el reviewer ochentoso más popular donde popularidad es la suma de los likes de los ratings que publicó y "ochentoso" porque solo se toman las reviews para películas de los 80s.

Pregunta 9:

Indicar cuál es la función del **Stub** en un ambiente cliente servidor usando gRPC.

Respuesta

El Stub funciona de proxy a un objeto remoto en el cliente. Implementa los mismos métodos que la interfaz remota. Su objetivo es recibir las invocaciones del cliente y pasarlas por la red (el método invocado y los parámetros) y esperar los resultados para leerlos y transmitirlos al cliente.

Pregunta 11:

¿Cuál es la utilidad del **Batch Loader** en GraphQL?

Respuesta

Batch Loaders es un concepto que se crea para mitigar el riesgo de caer en el problema de $n + 1$ queries.

Este problema se da cuando yo solicito un listado de elementos que requiere para resolver parte de su contenido hacer una query más por cada elemento del sistema.

Al registrarle un Batch Loader a un Field lo que hace GraphQL es acumular los pedidos a ese campo (que generalmente es por id) y en vez de hacer n llamados con 1 elemento, hace 1 llamado con los n elementos en un array. De esta manera quien implemente puede resolver la respuesta de una manera más eficiente

2023 2C 2

Pregunta 1

Indicar 3 características que deben cumplir las APIs REST

Respuesta

1. Arquitectura cliente-servidor, el API funciona con un modelo request-response, donde el cliente hace pedidos y el servidor los responde.
2. Stateless: No se guarda estado en la conexión. Esto significa que cada pedido tiene toda la información necesaria para que el servidor pueda responderlo, incluyendo, por ejemplo, autenticación.
3. Cacheable: Los recursos deben poder ser cacheables para evitar uso innecesario de la red. Por ejemplo, para una imagen un servidor HTTP me puede decir "esta es válida por 1 hora", tal que no tenga que volver a hacer el pedido por la misma imagen en ese tiempo.

Pregunta 2

Una instancia de la siguiente clase se quiere usar para compartir entre threads, los potenciales valores de estados de una aplicación.

Indicar, si existen, cuales son los riesgos de utilizar dicha instancia y las posibles soluciones para mitigar los problemas indicados.

```
public class States {  
    private final String[] states = new String[] { "UNK", "NEW", "RUNNING", "DONE" };  
    public String[] getStates() {  
        return states;  
    }  
}
```

Respuesta

El problema con este código es que por más que la variable `states` sea inmutable, el valor a la que esta apunta (un array en el heap) no lo es. Un thread podría modificar el estado, por ejemplo con `myStates.getStates()[0] = "Alfajor"`.

La idea de este código era usar objetos inmutables, el problema es que un campo sea final no necesariamente significa que sus contenidos sean inmutables. Para corregir esto, podemos reemplazar el `String[]` por, por ejemplo, una `List<String>` creada con `Collections.immutableList()`, tal de asegurarnos que ningún thread pueda modificar esa lista.

Pregunta 3

Que significa que un sistema distribuido sea elástico

Respuesta

La elasticidad de un sistema distribuido refiere a su capacidad de agregar o remover nodos sin tener que apagar y volver a prender el sistema.

Esto no es trivial, ya que, por ejemplo en una base de datos distribuidas, agregar un nodo significa que hay que asignarle particiones para ser dueño o réplica, o al remover un nodo hay que tomar las particiones de las que este era dueño y promover una réplica a ser el nuevo dueño.

Pregunta 4

En hazelcast por defecto ¿Cómo se selecciona al nodo dueño de una partición?

Mi respuesta

Para la coordinación, asignación de trabajo y particiones, Hazelcast elige como coordinador al nodo de mayor antigüedad en el cluster. Este nodo mantiene y distribuye la tabla de particiones, indicando qué nodo es owner o backup de cada partición. Cuando un nodo se suma o se va, el coordinador actualiza y replica la tabla a todos. Si el coordinador falla, el siguiente nodo más antiguo toma el rol y recalcula la tabla de particiones.

Pregunta 5

Explique el concepto de *Split Brain*. ¿Puede ocurrir utilizando el framework Hazelcast? Si es así, ¿cómo?

Mi respuesta

Split Brain ocurre cuando un cluster se divide en dos o más partes aisladas que no se pueden ver mutuamente, típicamente por fallos en la red o caída de nodos. Esto causa que, en breve, "el grafo deje de ser conexo". Esto es un problema, porque cada parte aislada no puede saber si la otra parte sigue operando o no, entonces las partes sin master eligen un nuevo master, y lo que antes era un solo sistema autónomo ahora son varios.

El problema ocurre cuando se recupera el estado de la red, ya que hasta entonces cada parte aislada puede haber aceptado pedidos de escrituras, resultando en estados distintos. Entonces, ¿Cómo debemos merge-ear los estados distintos? ¿Quién será el nuevo nodo master del cluster reagrupado?

Hazelcast tiene distintas formas de resolver esto. Por ejemplo, uno puede mantener el estado de la parte del cluster con más nodos, y que el nuevo master sea el nodo con mayor antigüedad.

Pregunta 6

Explicar la estrategia de Cassandra realiza la escritura y replicación de un dato

Respuesta

Cassandra usa Consistent Hashing para distribuir los datos. Esto le permite, al momento de querer acceder a un dato, encontrar en qué nodo está usando nada más un hash de la partition key.

Notar que esto permite que cualquier nodo responda un pedido de escritura, a diferencia de bases de datos como MongoDB donde las escrituras solo las puede realizar el master. En Cassandra no hay un nodo master.

Pregunta 7

Que es el factor de replicación y como puede afectar a la latencia y consistencia a la hora de realizar escrituras en una base de datos NoSQL

Respuesta

El factor de replicación indica cuantas réplicas se harán de un dato/partición. Esto puede afectar la latencia porque, ¿Cuando se considera que un dato está escrito?

Si un dato está escrito cuando lo recibe el nodo que tomó el pedido, entonces si dicho nodo crashea antes de mandarlo a otro, el dato se habrá perdido, por más que el cliente recibió una respuesta que lo hace creer que el dato está escrito. En este caso también puede ocurrir que el cliente intente leer el dato recién escrito y el sistema responda que el dato no existe. La latencia de respuesta será muy baja, pero la consistencia será mala.

Entonces uno puede decir que el dato está escrito cuando tiene por lo menos una réplica, o cuando ya fue escrito en todas las réplicas que debe tener. Este último caso aumentará la latencia de escritura (tardará substancialmente más tiempo en dar el OK al cliente), pero dará mejor consistencia, ya que al recibir el cliente el OK ya sabemos que dicho dato está replicado y listo para ser leído.

Pregunta 8

Indicar si es correcto que en el siguiente código:

```
public class Servlet {
    private Object lockCounterLock;
    private int counter;
    public void responseRequest(Request request) {
        ...
        synchronized (lockCounterLock) {
            ...
            counter ++;
        }
        ...
    }

    public synchronized int getCount(){
        return counter;
    }
}
```

La instrucción

```
synchronized (lockCounterLock) {
    ...
}
```

indica que ningún otro código de la clase puede modificar la variable lockCounterLock
(VERDADERO O FALSO)

Respuesta

Falso

Pregunta 10

Supongamos un dataset de **molinetes de estaciones de subte** de una ciudad que modela sus datos de la siguiente forma:

- **id**: identificador autogenerado
- **estacion**: Nombre de la estación donde se encuentra el molinete.
- **linea**: **Nombre de la línea de subte a la cual pertenece la estación donde se encuentra el molinete**
- **fechaHora**: Fecha y hora de la medición del molinete
- **pasajeros**: Cantidad de pasajeros que pasaron por el molinete en la fecha indicada

donde existen hasta 24 registros por día para cada molinete (uno por hora).

Suponiendo que se quiere utilizar un proceso mapreduce (que puede ser una cadena de procesos).

En este proceso se recibe en el mapper inicial el id de la medición como clave y el resto de los datos del registro como valor.

Se pide sin codificar, indique en palabras

Para cada job en la cadena

- ¿Cuáles serían el comportamiento del Mapper?
- ¿Cuál es la clave y el valor que emite mapper?
- ¿Cuál sería el comportamiento del Reducer?
- ¿Cuál es la clave y el valor que emite Reducer?
- Indicar si hay que hacer un procesamiento posterior de los resultados

Para el caso en que se quiera resolver la siguiente consulta:

Estación con más pasajeros de cada línea de subte para el año 2021, indicando el nombre de la estación, el nombre de la línea y la cantidad total de pasajeros que pasaron por todos los molinetes de esa estación para el año 2021. El orden es alfabético por nombre de la línea.

Respuesta

Escribo un pseudocódigo para ayudar la explicación:

```
function map(id, estacion, linea, fechaHora, pasajeros) {
  if (fechaHora >= "01/01/2021" && fechaHora <= "31/12/2021") {
    emit(Pair(estacion, linea), pasajeros);
  }
}

function reduce(estacion_y_linea, values) {
  let total = 0;
  for elem in values {
    total += elem;
  }

  return (estacion_y_linea, total);
}
```

El mapper toma cada dato de molinete y, si la fecha de este está dentro del año 2021, emite un par clave-valor, donde la clave es compuesta, una tupla de la estación y la línea, y el valor es la cantidad de pasajeros. El reducer luego suma todos los valores de pasajeros para cada par estación-línea.

Este mapreduce produce un par (estación-línea, totalPasajeros) por cada estación-línea presente. En breve, calcula el total de pasajeros para cada estación-línea. A continuación se requiere hacer un post-procesamiento, donde se toma cada uno de estos registros y nos quedamos, para cada línea, con la estación que tenga la mayor cantidad de pasajeros. Esas tuplas (línea, estación, totalPasajeros) luego deben ser ordenadas alfabéticamente por el nombre de la línea para producir el resultado final.

Notar que si una estación no tiene ningún registro en el año 2021, esta no podrá aparecer en la lista de la salida.

Preguntas Bruzo

¿Por qué son preferibles los objetos inmutables en ambientes concurrentes?

Los objetos inmutables son preferibles en ambientes concurrentes porque son thread-safe por naturaleza: su estado no cambia, por lo que pueden ser compartidos entre hilos sin riesgo de race conditions ni necesidad de sincronización.

¿Cómo se produce un LiveLock?

El livelock es similar al deadlock en que dos hilos no pueden avanzar, pero no quedan bloqueados, sino que siguen activos cediéndose mutuamente el paso de forma indefinida. El resultado es un bucle infinito de "tomo y libero", causado por intentos repetidos de ser "educados".

